mindPal

Measuring Internal Product Attributes: Software Size

1. SOFTWARE SIZE: LENGTH (LOC)

The MindPal system was measured for code length across its two primary files. The server.js file contains approximately 280 non-commented source lines of code (NCLOC) and approximately 35 commented source lines of code (CLOC), giving a total LOC of 315 for the backend. The public/index.html file contains approximately 420 NCLOC and 15 CLOC, giving a total LOC of 435 for the frontend. The .env configuration file contributes 6 LOC and the package.json contributes 14 LOC.

Total system LOC = 315 + 435 + 6 + 14 = 770 LOC. Total NCLOC = 280 + 420 + 6 + 14 = 720. Total CLOC = 35 + 15 = 50. Comment density = CLOC / LOC = 50 / 770 = 6.5%.

The comment density of 6.5% reflects a deliberate design choice. The system prompts embedded in server.js are self-documenting by nature since each therapy mode name and its clinical content is self-explanatory. The primary comments appear in the server startup section, the route handler boundaries, and the .env file where user instruction is required. This is consistent with the course observation that comment density is a meaningful quality indicator when measured in context rather than in isolation.

The LOC measure confirms that MindPal is a small to medium sized system by conventional standards. The backend at 315 LOC is well within the course guideline that modules should ideally contain fewer than 200 LOC per functional unit, with the largest single functional unit being the SYSTEM_PROMPTS object which, while large in character volume, constitutes a data structure rather than executable logic and would not be counted under the executable statements only LOC variation.

2. HALSTEAD'S SOFTWARE SCIENCE APPLIED

Halstead's measures were applied to the most complex function in MindPal, the /api/chat route handler in server.js, to quantify its length, volume, difficulty, and estimated development effort.

The distinct operators identified in the /api/chat function include: require, const, app.post, async, await, if, return, try, catch, JSON.stringify, fetch, method, headers, body, res.status, res.json, console.error, map, filter, join, process.env, and the assignment operator. This gives a count of approximately 23 distinct operators, so $\mu 1 = 23$.

The distinct operands include: messages, mode, apiKey, systemPrompt, SYSTEM_PROMPTS, response, data, reply, req, res, GROQ_API_KEY, Content-Type, Authorization, model, max_tokens, temperature, choices, content, error, 400, 500, 800, 0.75, and llama-3.3-70b-versatile. This gives approximately 24 distinct operands, so $\mu 2 = 24$.

Program vocabulary: μ = μ1 + μ2 = 23 + 24 = 47.

The total occurrences of operators N1 is estimated at approximately 68 and total occurrences of operands N2 is estimated at approximately 52, giving program length N = N1 + N2 = 120.

Program volume: V = N × log2(μ) = 120 × log2(47) = 120 × 5.554 = 666.5 mental comparisons.

Estimated program length: N-hat = μ1 × log2(μ1) + μ2 × log2(μ2) = 23 × log2(23) + 24 × log2(24) = 23 × 4.524 + 24 × 4.585 = 104.05 + 110.04 = 214.09.

The actual length N of 120 is considerably lower than the estimated length of 214, which reflects the fact that the function makes heavy use of modern JavaScript shorthand constructs such as optional chaining, array methods, and template literals that pack significant logical content into compact syntax. This is consistent with the Halstead observation that higher-level languages reduce physical length relative to estimated length.

Program difficulty: D = (μ1 / 2) × (N2 / μ2) = (23 / 2) × (52 / 24) = 11.5 × 2.167 = 24.92.

Effort: E = V × D = 666.5 × 24.92 = 16,606 elementary mental discriminations.

Development time estimate: T = E / β = 16,606 / 18 = 922 seconds, approximately 15 minutes of focused development time for this single function.

Remaining bugs estimate: B = E to the power of 2/3 divided by 3000 = 16,606 to the power of 0.667 divided by 3000 = 659.8 / 3000 = 0.22 bugs.

The Halstead bug estimate of 0.22 for the /api/chat function is consistent with the observed development experience. The function required one significant debugging session (the API key not loading from .env) which corresponds to approximately the predicted defect level. The low bug count reflects the function's clean, linear structure despite its apparent complexity.

3. FUNCTION POINT ANALYSIS

The MindPal system was analysed using the IFPUG Function Point counting method to provide a language-independent measure of functional size.

External Inputs (EI): The user text message submission constitutes one EI, as it enters the application boundary and triggers maintenance of the conversation history ILF. The mood check-in widget submission constitutes a second EI, as it enters a numeric value that updates the conversation history. The therapy mode tab selection constitutes a third EI, as it changes the mode variable that controls system behaviour. The quick-access tool button activations (breathing, grounding, thought record, etc.) collectively constitute a fourth EI category. Total EI count = 4.

External Outputs (EO): The AI therapy response displayed in the chat bubble constitutes one EO, as it presents derived information generated through processing logic (the Groq API call with

system prompt application). The session timer display constitutes a second EO, as it presents a calculated value (seconds converted to minutes and seconds format) derived from a running computation. The toast error notification constitutes a third EO, as it presents a derived status message based on server response processing. Total EO count = 3.

External Inquiries (EQ): The health check endpoint /health constitutes one EQ, as it retrieves and returns server status without mathematical processing or ILF maintenance. Total EQ count = 1.

Internal Logical Files (ILF): The conversation history array maintained in frontend memory constitutes one ILF. It is a user-identifiable group of logically related data (message objects with role and content) maintained within the application boundary through the message submission elementary processes. Total ILF count = 1.

External Interface Files (EIF): The Groq API constitutes one EIF, as it is a logically related group of data and control information referenced by the application but maintained within the boundary of the external Groq system. The .env configuration file constitutes a second EIF referenced by server.js but maintained as a separate external configuration artifact. Total EIF count = 2.

Unadjusted Function Point Count using average complexity weights: UFC = 4 × NEI + 5 × NEO + 4 × NEQ + 7 × NEIF + 10 × NILF UFC = 4 × 4 + 5 × 3 + 4 × 1 + 7 × 2 + 10 × 1 UFC = 16 + 15 + 4 + 14 + 10 UFC = 59 unadjusted function points.

Value Adjustment Factor (VAF): The 14 technical complexity factors were assessed for MindPal as follows. Data communications (F1) was rated 5 because the entire application depends on communication facilities (Groq API over HTTPS). Performance (F3) was rated 4 because response speed is a key user experience requirement for emotional support interactions. On-line data entry (F6) was rated 5 because the entire interface is real-time text entry. End-user efficiency (F7) was rated 5 because the toolbar, starter pills, and mood widget were all specifically designed for maximum efficiency. Complex processing (F9) was rated 4 because the system prompt selection and clinical reasoning involve significant processing logic. Reusability (F10) was rated 3 because the Node.js backend is designed to be portable across hosting environments. Multiple sites (F13) was rated 3 because the system is designed to run on any hosting platform. Facilitate change (F14) was rated 4 because the SYSTEM_PROMPTS object is designed for easy extension. All other factors (F2, F4, F5, F8, F11, F12) were rated between 0 and 2.

Approximate sum of Fi = 5 + 0 + 4 + 0 + 0 + 5 + 5 + 0 + 4 + 3 + 1 + 1 + 3 + 4 = 35.

VAF = 0.65 + 0.01 × 35 = 0.65 + 0.35 = 1.00.

Final adjusted Function Point: FP = UFC × VAF = 59 × 1.00 = 59 function points.

Using the Jones LOC per FP table for JavaScript at approximately 55 SLOC per FP, the predicted code size would be 59 × 55 = 3,245 SLOC. The actual measured NCLOC of 720 is lower than this prediction, which is expected because MindPal is a single-purpose application

that reuses the Groq API for all AI processing rather than implementing complex logic internally. The difference between predicted and actual size reflects the high degree of functionality delegated to the external API, which reduces the internal code footprint while maintaining the functional capability counted in the FP analysis.

4. FEATURE POINT ANALYSIS

MindPal includes a significant algorithmic component in the form of the therapy mode selection and system prompt construction logic, making Feature Point analysis more appropriate than standard Function Point analysis for capturing the full functional complexity of the system.

In addition to the five standard FP categories counted above, the following algorithms were identified in MindPal.

Algorithm 1: Therapy mode to system prompt mapping. The server receives a mode string, performs a key lookup in the SYSTEM_PROMPTS object, and selects the appropriate clinical prompt. This is a bounded computational process that transforms an input value into a specific output configuration.

Algorithm 2: Conversation history construction. The frontend builds the messages array by appending each new user message and AI response in the correct role-ordered format required by the Groq API. This is a bounded computational process governing how state is accumulated and transmitted.

Algorithm 3: Mood score to therapeutic response calibration. The AI system prompt instructs the model to interpret mood scores in three distinct ranges (1-3, 4-6, 7-10) and adjust its clinical approach accordingly. This is a bounded computational process embedded in the prompt logic.

Total algorithm count NA = 3.

Unadjusted Feature Count: UFeC = 4 × NEI + 5 × NEO + 4 × NEQ + 7 × NEIF + 7 × NILF + 3 × NA UFeC = 4 × 4 + 5 × 3 + 4 × 1 + 7 × 2 + 7 × 1 + 3 × 3 UFeC = 16 + 15 + 4 + 14 + 7 + 9 UFeC = 65 unadjusted feature points.

The feature point count of 65 is approximately 10% higher than the function point count of 59, consistent with the course observation that feature points typically produce counts 20 to 35 percent higher than function points for systems with high algorithmic complexity. The relatively modest increase of 10% reflects the fact that MindPal delegates most of its algorithmic processing to the external Groq API rather than implementing it internally, placing it between a pure business information system and a complex algorithmic system on the feature point scale.

5. OBJECT POINT ANALYSIS

Object Point analysis was applied to MindPal to estimate functional size from the perspective of screens and reports, which is particularly useful for early-stage planning before detailed code exists.

Screens identified in MindPal:

The welcome screen with approach cards and starter pills constitutes one screen. It contains multiple views (approach cards, starter pills, welcome text) and references no server-side data tables, making it a simple screen with a weight of 1.

The main chat interface constitutes one screen. It contains the message display area, input box, mode bar, and toolbar. It references the conversation history (one logical data source) with moderate view complexity, making it a medium screen with a weight of 2.

The mood check-in widget constitutes one screen component. It contains 10 interactive buttons and two label views with one logical data source (the mood value), making it a simple screen with a weight of 1.

Total screen object points = 1 × 1 + 1 × 2 + 1 × 1 = 4.

Reports identified in MindPal:

The AI therapy response bubble constitutes a report as it presents output information to the user. It references one data source (the Groq API response) with one primary view (the response text), making it a simple report with a weight of 2.

The session timer display constitutes a report presenting a calculated time value. It references one data source (the session counter) with one view, making it a simple report with a weight of 2.

The error toast notification constitutes a report presenting a status message. It references one data source (the server error response) with one view, making it a simple report with a weight of 2.

Total report object points = 3 × 2 = 6.

Total Object Points = 4 + 6 = 10.

No acquired third-generation language components were used. The Groq API is an external service rather than an acquired 3GL component. Reuse from previous projects was estimated at 0% as MindPal was developed from scratch.

Effort estimate using the OP formula with average developer experience (PROD = 13): Effort = OP / PROD = 10 / 13 = 0.77 person-months.

This estimate of approximately 0.77 person-months (roughly 3 weeks of full-time development) is consistent with the actual development time experienced, confirming that the Object Point method produced a reasonable early-stage effort estimate for MindPal.

6. USE-CASE POINT ANALYSIS

MindPal's functional requirements can be expressed as use cases, enabling Use-Case Point analysis following the Karner method.

Actors identified:

The end user (person experiencing mental health distress interacting through the browser GUI) is a complex actor with weight 3. The Groq API (external system interacting through the HTTPS REST protocol) is an average actor with weight 2. The .env configuration system (data store accessed by the server) is an average actor with weight 2.

Unadjusted Actor Weight: UAW = 3 + 2 + 2 = 7.

Use Cases identified:

Use Case 1: Submit a therapy message and receive an AI response. This involves entering text, pressing send, the frontend appending to history, the server receiving the POST request, the server looking up the system prompt, the server calling Groq, the server returning the reply, and the frontend displaying the bubble. This involves more than 7 transactions and is a complex use case with weight 15.

Use Case 2: Select a therapy mode. This involves clicking a mode tab, the JavaScript updating the active mode variable, the visual state of the tab changing, and an optional mode-change message being appended to the chat. This involves 3 to 4 transactions and is a simple to average use case with weight 5.

Use Case 3: Submit a mood check-in. This involves clicking the mood check-in tool button, the widget appearing, the user clicking a number, the mood value being appended to history, the widget being removed, and the AI response being triggered. This involves 6 transactions and is an average use case with weight 10.

Use Case 4: Trigger a therapy tool (breathing, grounding, thought record etc.). This involves clicking a tool button, the tool prompt being constructed, the prompt being appended to history, the server call being made, and the AI response being displayed. This involves 5 transactions and is an average use case with weight 10.

Use Case 5: Session timer operation. This involves the timer starting on first message, incrementing every second, and displaying in the header. This involves 3 transactions and is a simple use case with weight 5.

Unadjusted Use Case Weight: UUCW = 15 + 5 + 10 + 10 + 5 = 45.

Unadjusted Use Case Points: UUCP = UAW + UUCW = 7 + 45 = 52.

Applying the same VAF of 1.00 calculated in the Function Point section: UCP = UUCP × VAF = 52 × 1.00 = 52 use-case points.

The use-case point count of 52 is comparable to the function point count of 59, confirming consistency across size measurement methods. The slight difference reflects the fact that the use-case method captures the interaction complexity of the user flows, while the function point method captures the data and transaction boundary complexity of the system, and the two perspectives emphasise slightly different aspects of the same functional size.

7. SOFTWARE REUSE

MindPal's architecture makes deliberate use of reuse at multiple levels, following the reuse classification framework from the course.

Reused verbatim components (zero lines modified): The Express.js framework, CORS middleware, dotenv library, and node-fetch library were all reused verbatim from the npm registry without any modification. These constitute externally reused components that contribute significantly to the functional capability of the system without adding to the internal LOC count.

The Groq API's OpenAI-compatible endpoint format was reused verbatim from the standard OpenAI API request structure, meaning the message array format, model parameter, and response structure required no custom adaptation from the developer.

Reuse level: Approximately 4 npm packages were reused out of a total of approximately 6 distinct functional components in the system (express, cors, dotenv, node-fetch, server logic, frontend logic). Reuse level = 4 / 6 = 67%.

Reuse density: The npm node_modules directory contains 75 packages installed automatically as dependencies of the four directly referenced packages. This means 75 packages of reused code support the execution of 720 NCLOC of original code, giving a reuse density that strongly favours reused components over original development.

Reuse frequency: Every API call made by the system references reused components. The fetch function references node-fetch. The routing references express. The key loading references dotenv. Every transaction in the system involves at least one reused component, giving a reuse frequency of 100% across all elementary processes.

The high reuse level, density, and frequency are consistent with the course principle that measuring size must account for reused products. If MindPal's size were measured purely by original NCLOC of 720, it would significantly understate the functional capability delivered, because the majority of the system's capability is provided by reused and external components. The Function Point count of 59, which is independent of implementation language and reuse level, provides a more accurate representation of the delivered functional size than LOC alone